Sir Syed University of Engineering & Technology Computer Engineering Department

Object Oriented Programming (CE-225L)

Semester: 3rd

Batch: Fall 2024

Section: A

Project Report

Loan management system

Submitted by:

Group Member	Roll Number
Shahzeb Shahid	2024F-BCE-029
Mosa Raza	2024F-BCE-038
M. Shaheer Asif	2024F-BCE-040

Submitted To:

Engr. Samina Noreen

Table of Contents

1. Abstract----------3
2. Introduction----------3
   2.1 Background----------3
3. Definitions and Acronyms----------4
4. Proposed System----------5
5. Feasibility Report----------6
   5.1 General Requirement Feasibility----------6
   5.2 Problems in the Current System----------6
6. Technical Feasibility----------7
7. Technical Description----------8
8. Graphical User Interface (GUI)----------11
9. Project Design Description----------15
   9.1 Purpose----------15
   9.2 Scope----------15
10.Scope of the Development Project ----------15
10.1 Database Connectivity Tier----------15
11.Software Requirement Specification (SRS) ----------15
   11.1 Hardware Requirements ----------15
   11.2 Software Requirements----------15
12.Data Flow Diagrams (DFD) ----------16
13.Conclusion and Recommendations----------17
14.Acknowledgements----------18
15.References----------19

1. Abstract

The Loan Management System is a desktop application developed in **Java SE** using **Swing** for the graphical interface. It follows **Object-Oriented Programming (OOP)** principles and provides a step-by-step wizard to collect borrower information, loan details, guarantors, and terms. The system generates an affidavit at the end, making the process structured and user friendly.

Currently, the system runs in-memory without database support, designed mainly for academic and demonstration purposes. It shows how Java Swing can be used to build a complete loan intake workflow, and it can be extended in the future with features like database storage, dynamic interest calculation, and multi-user access.

2. Introduction

[2.1] BACKGROUND

Today, managing loans properly is very important. If loan records, borrower details, guarantor information, and repayment schedules are handled manually, it takes a lot of time and mistakes can easily happen. To solve these problems, we created a **Loan Management System (LMS)** — a simple and user-friendly desktop application made in Java.

The LMS works as a single place where all loan-related information can be stored and managed. It helps administrators keep track of borrower details, loan amounts, mortgage items, and guarantors. It also guides applicants' step by step to enter their information correctly.

This project uses **Object-Oriented Programming (OOP)** concepts like classes, inheritance, and encapsulation. These make the system modular, reusable, and easy to maintain.

The project fulfills the academic goals of the **CE-225L Object Oriented Programming**

3. Definitions and Acronyms

TERM	DEFINITION
LMS	Loan Management System – The desktop application developed for this project
OOP	Object-Oriented Programming – A way of programming using classes and objects
GUI	Graphical User Interface – The visual part of the software where users enter and see information
CNIC	Computerized National Identity Card – Used for borrower and guarantor identification
Affidavit	A written statement generated by the system to confirm loan details
Mortgage	Property or valuable item given as security for the loan
Guarantor	A person who promises to repay the loan if the borrower fails
SRS	Software Requirement Specification – Document that explains hardware and software needs of the project
DFD	Data Flow Diagram – Diagram that shows how information moves through the system

4. Proposed System

The **Loan Management System (LMS)** is designed to simplify and accelerate the loan application process using **Java SE** and **Object-Oriented Programming (OOP)** principles. The system provides a structured workflow that collects borrower information, verifies eligibility, and generates affidavits for loan approval. By integrating multiple modules into a single desktop application, the LMS reduces manual effort, minimizes errors, and ensures a user-friendly experience.

Main Features

- **Borrower Registration:**
  Captures personal information such as name, city, CNIC, and education.
- **Income Details:**
  Records employment salary or business income for financial assessment.
- **Past Loan History:**
  Tracks previous loans, repayment status, and outstanding balances.
- **Living Situation & Assets:**
  Notes house and car ownership, rental, or loan status along with asset values.
- **Loan & Mortgage:**
  Allows borrowers to request loans against mortgage items such as property, vehicles, or jewelry.
- **Guarantors:**
  Requires two guarantors with valid credentials and no criminal record.
- **Terms & Conditions:**
  Displays interest rates, penalties, repayment rules, and bank transaction details.
- **Affidavit Generation:**
  Produces a formal affidavit containing borrower, guarantor, and mortgage details for approval.

Workflow

1. **Input:** Borrower enters personal, financial, and guarantor details.
2. **Verification:** The system checks eligibility based on category, income, and mortgage value.
3. **Approval:** Borrower reviews and accepts terms and conditions.
4. **Output:** A structured affidavit is generated for loan approval.

5. Feasibility Report

5.1 General Requirement Feasibility

- **Functional Feasibility:**
  The system successfully implements all required intake steps, including personal information, income details, loan request, guarantor verification, and affidavit generation. Each module is integrated into the wizard-style GUI, ensuring completeness of the loan application workflow.
- **Technical Feasibility:**
  The application is developed in **Java SE** using **Swing** for the graphical interface. It can run on any standard personal computer without requiring specialized hardware. The modular OOP design ensures maintainability and future scalability.
- **Operational Feasibility:**
  The wizard-style flow guides the user step by step, making the system intuitive and user-friendly. Borrowers can easily navigate through panels, reducing training requirements and minimizing input errors.

5.2 Problems in the Current System

- **No Database Persistence:**
  The system currently operates in memory only. Once the application is closed, all borrower and loan details are lost. This limits practical use in real financial institutions.
- **Single-User Limitation:**
  The application supports only one user session at a time. There is no provision for multiple users, role-based access, or administrative control.
- **Static Interest and Penalty Values:**
  Interest and penalty rates are hard-coded in the system. They remain fixed regardless of loan type, duration, or borrower category, reducing flexibility.
- **No Export/Print Functionality:**
  Although the system generates affidavits, they cannot be saved as files or printed directly. This restricts usability in real-world scenarios where documentation is required.

6. Technical Feasibility

- **Platform:**
  Developed in **Java SE 8+** using **Swing and AWT** for the GUI. Portable and runs on any standard computer with Java installed.
- **Libraries:**
  Uses only **standard JDK libraries** (Swing, AWT, util, text), keeping the project lightweight and easy to maintain.

- **OOP Principles:**
  - Person, Loan Taker, and Loan Details classes show **inheritance and encapsulation**.
  - The Gui class uses **Card Layout** to manage multiple panels step-by-step.
  - Input fields, radio buttons, and combo boxes make data entry structured and user friendly.
- **Complexity:**
  Moderate — multiple GUI panels, validation, and affidavit generation are manageable for students while still reflecting real-world application design.
- **Extensible:**
  Can be extended with **JDBC/ORM** for database persistence, dynamic interest calculation, and multi-user support, making it scalable for institutional or professional use.

7. Technical Description

7.1 Classes:

- **Person (Base Entity):**

Base entity with borrower details (name, city, phone, CNIC, education)

```
public class Person {
    private String name;
    private String city;
    private String phone;
    private String cnic;
    private String education;
    // Constructor
    public Person(String name, String city, String phone, String cnic, String education) {
        this.name = name;
        this.city = city;
        this.phone = phone;
        this.cnic = cnic;
        this.education = education;
    }
    // Getters and Setters (Encapsulation)
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    // ... other getters and setters
}
```

- **Loan Taker (Inheritance):**

Extends Person, adds mortgage, income, past loan history, and living situation.

```
public class LoanTaker extends Person {
    private double income;
    private String mortgageItem;
    private double mortgageValue;
    private String pastLoanHistory;
    private String livingSituation;

    public LoanTaker(String name, String city, String phone, String cnic, String education,
                     double income, String mortgageItem, double mortgageValue,
                     String pastLoanHistory, String livingSituation) {
        super(name, city, phone, cnic, education); // Inheritance
        this.income = income;
        this.mortgageItem = mortgageItem;
        this.mortgageValue = mortgageValue;
        this.pastLoanHistory = pastLoanHistory;
        this.livingSituation = livingSituation;
    }
    // Encapsulation with getters/setters
    public double getIncome() { return income; }
    public void setIncome(double income) { this.income = income; }
    // ... other getters and setters
}
```

- **Loan Details (Encapsulation):**

Encapsulates loan amount, duration, interest rate, penalty, and payment method.

```java
public class LoanDetails {
    private double loanAmount;
    private int durationMonths;
    private double interestRate;
    private double penalty;
    private String paymentMethod;

    public LoanDetails(double loanAmount, int durationMonths,
                       double interestRate, double penalty, String paymentMethod) {
        this.loanAmount = loanAmount;
        this.durationMonths = durationMonths;
        this.interestRate = interestRate;
        this.penalty = penalty;
        this.paymentMethod = paymentMethod;
    }
    // Example method: calculate total repayment
    public double calculateRepayment() {
        return loanAmount + (loanAmount * interestRate / 100) + penalty;
    }
}
```

- **GUI:**

  Swing framework managing panels with Card Layout, navigation buttons, validation, and affidavit generation.

7.2 Business Rules:

- Borrower with a criminal record → loan rejection.

```java
if (borrower.hasCriminalRecord()) {
    System.out.println("Loan Rejected: Criminal record found.");
}
```

- Loan limits enforced by category (student/job/business).

```java
if (borrower.getCategory().equals("Student") && loanAmount > 50000)
{
    System.out.println("Loan exceeds student limit.");
}
```

- Mortgage is mandatory and must be equal to or greater than loan amount.

```
if (loanTaker.getMortgageValue() < loanDetails.getLoanAmount())
{
    System.out.println("Loan Rejected: Mortgage value insufficient.");
}
```

- Two guarantors with clean records are required.

```
if (guarantors.size() < 2) {
    System.out.println("Loan Rejected: Two guarantors required.");
}
```

- Terms & Conditions must be accepted before approval.

```
if (!termsCheckBox.isSelected()) {
    System.out.println("Loan Rejected: Terms & Conditions not accepted.");
} else {
    System.out.println("Loan Approved: Terms & Conditions accepted.");
}
```

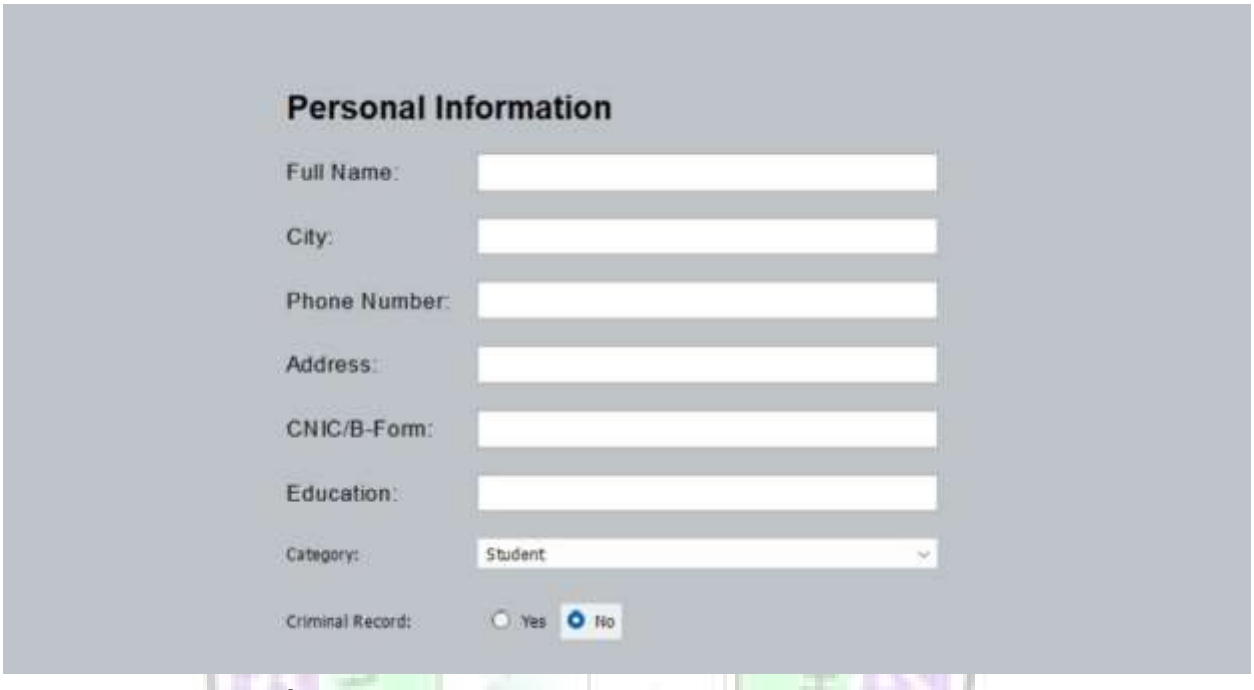
7.3 Technical Notes:

- Built in **Java SE 8+** using **Swing/AWT**.
- Lightweight, modular, and runs on any standard computer.
- Extensible with **JDBC/ORM** for database persistence, dynamic interest calculation, and multi-user support.

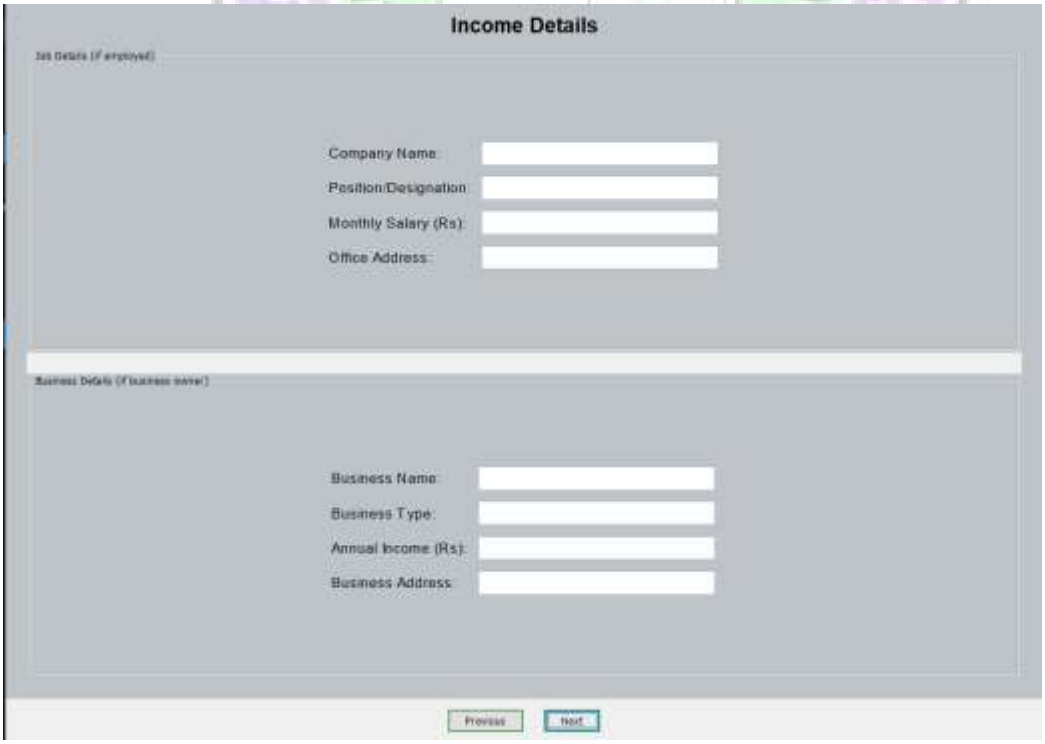
8. Graphical User Interface (GUI)

- **Panels**:

1. **Personal Information**

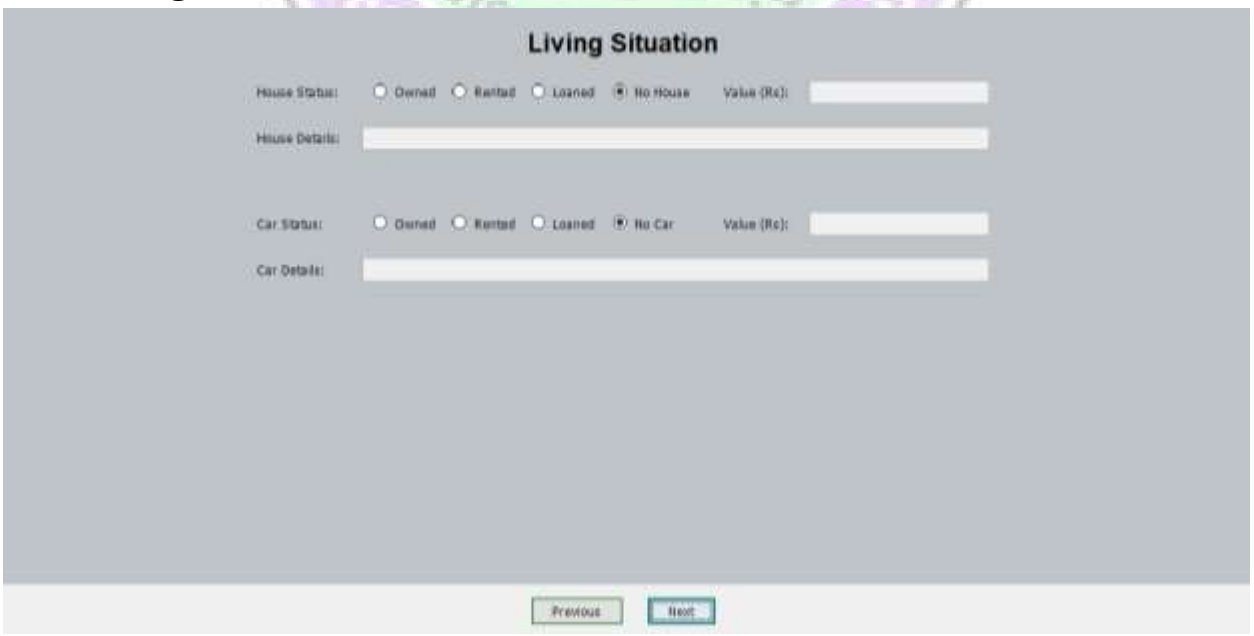


2. **Income Details**

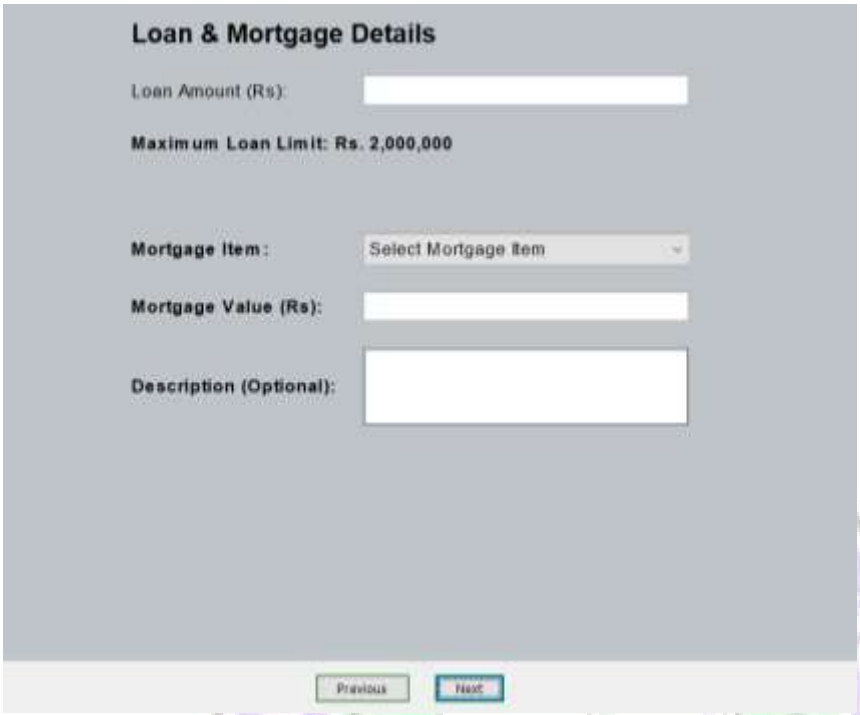


3. Past Loan History

4. Living Situation

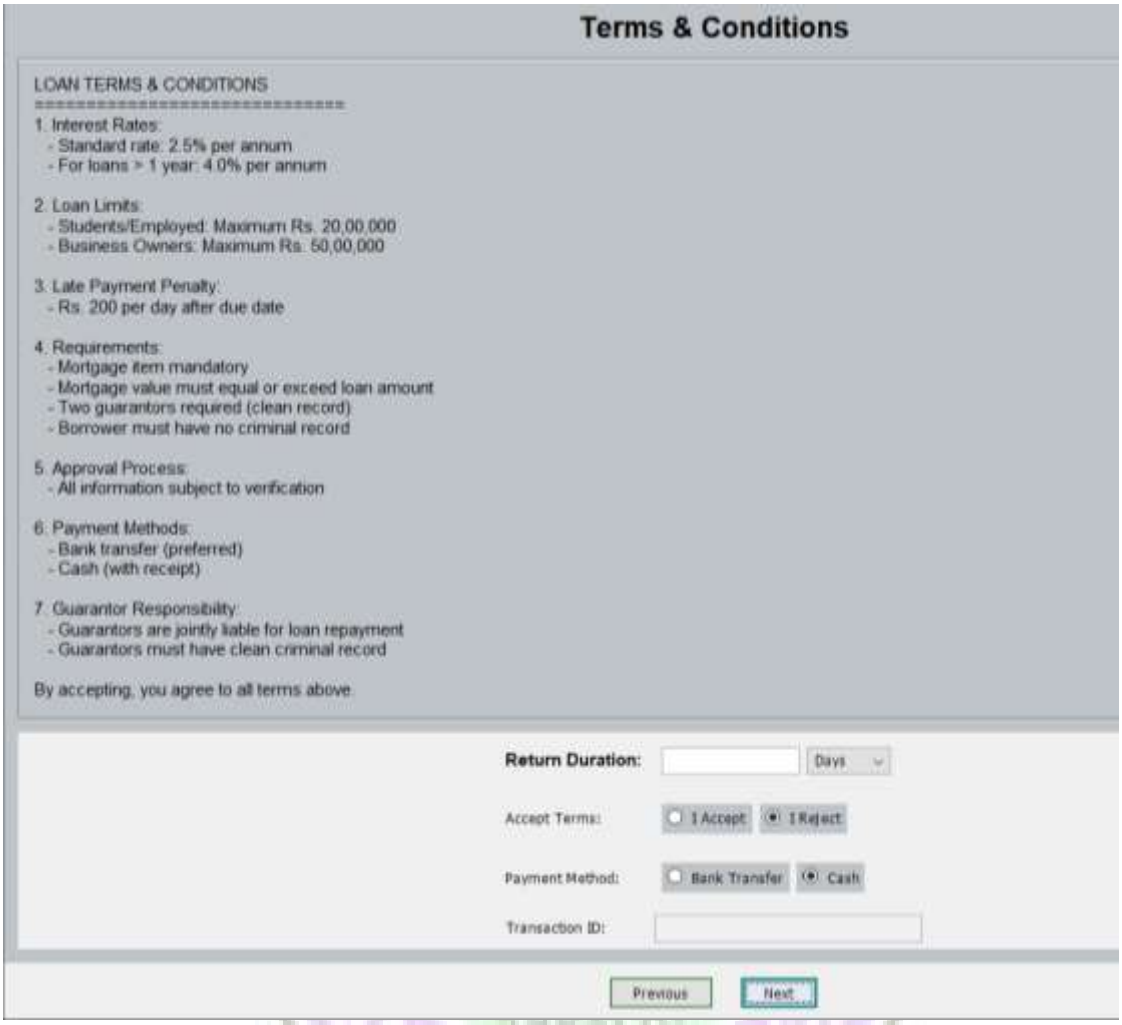


5. Loan & Mortgage

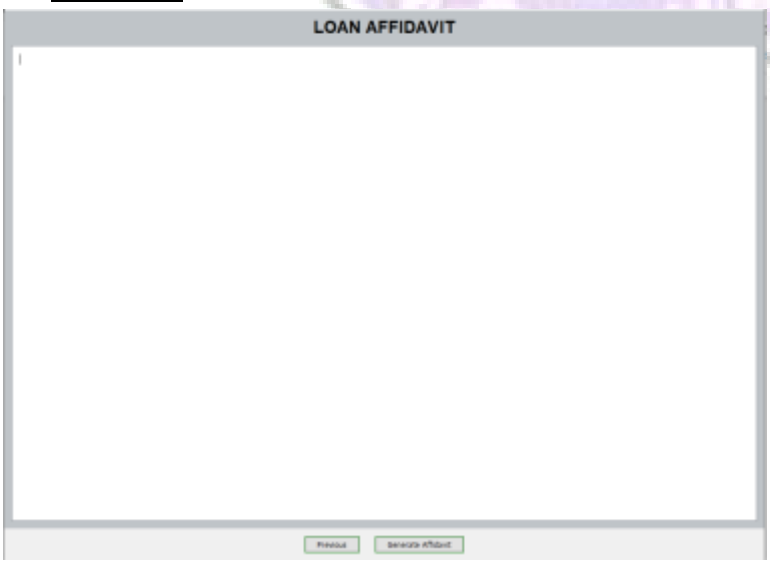


6. Guarantors

7. Terms & Conditions



8. Affidavit



9. Project Design Description

9.1 Purpose

To automate loan application intake, validate borrower details, and generate a formal affidavit for record-keeping. This reduces paperwork, minimizes errors, and streamlines loan approval.

9.2 Scope

Includes:

- Borrower data collection via step-by-step GUI
- Validation of rules (loan limits, guarantors, mortgage, criminal record)
- Loan limit enforcement by category
- Affidavit generation for approval

Excludes:

- Database persistence (in-memory only)
- Multi-user roles
- Reporting features

10. Scope of the Development Project

10.1 Database Connectivity Tier (Future)

Future development will include integration with **JDBC/Hibernate** to enable database persistence. This will support **CRUD operations** (Create, Read, Update, Delete) for loan applications, guarantor records, and policy management. Such connectivity will extend the system from an in-memory prototype to a scalable, multi-user application with reliable data storage.

11. Software Requirement Specification (SRS)

11.1 Hardware Requirements

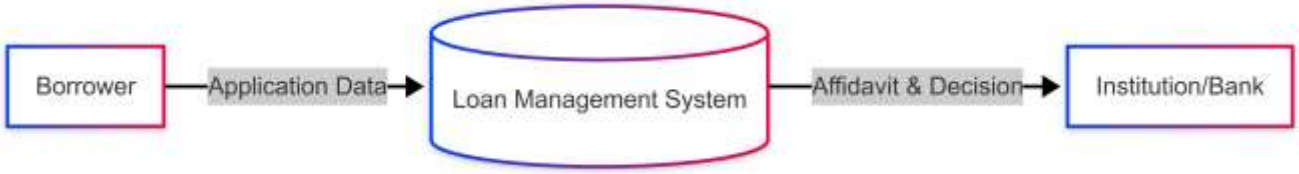
- Minimum: Dual-core CPU, 4 GB RAM.
- Recommended: Quad-core CPU, 8 GB RAM.

11.2 Software Requirements

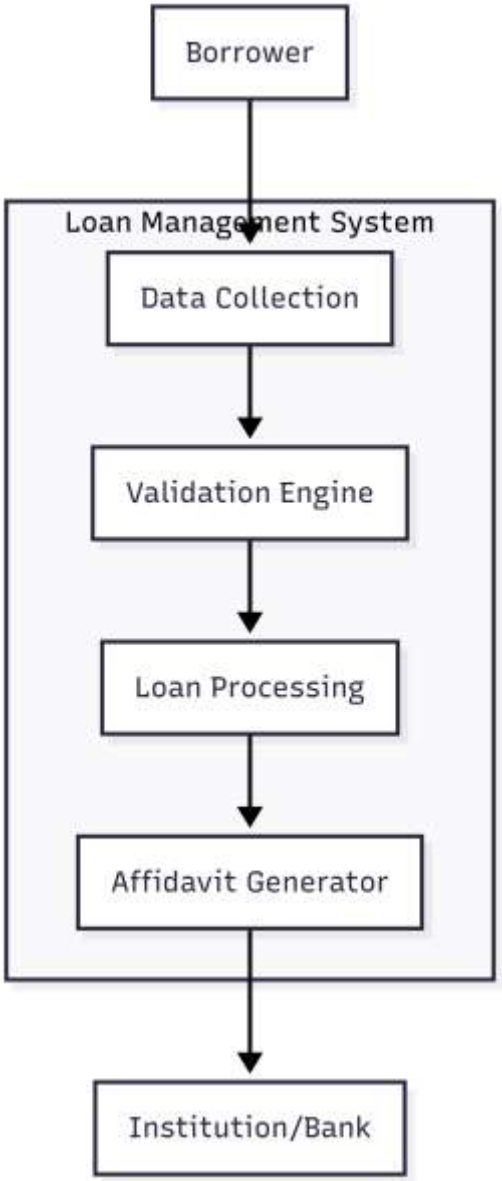
- Runtime: Java SE 8+
- OS: Windows/Linux/macOS
- IDE: IntelliJ/Eclipse/NetBeans

12. Data Flow Diagrams (DFD)

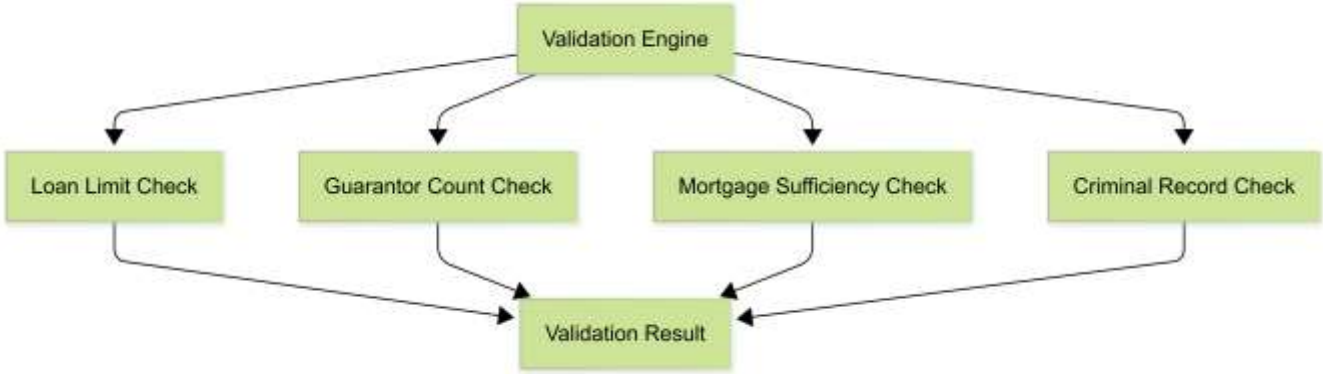
- **Level 0**
  Shows the high-level interaction between the user, system, and external entity.



- **Level-1 (Detailed flow)**
  Breaks down the internal workflow of the system step by step.



- **Level-2 (Validation engine breakdown)**
  Validation Engine → Loan Limit Check → Guarantor Count Check → Mortgage Sufficiency Check → Criminal Record Check → Validation Result



13. Conclusion

The Loan Management System successfully demonstrates how **Object-Oriented Programming (OOP)** principles and the **Java Swing framework** can be applied to automate loan application intake. The system collects borrower information in a structured manner, enforces business rules such as guarantor requirements, loan limits, and mortgage sufficiency, and generates a formal affidavit for institutional use.

Although the current version operates in memory without database persistence or multi-user roles, the design is **modular and extensible**, allowing future integration with JDBC/ORM, reporting features, and multi-user support.

14. Acknowledgement

We would like to express our heartfelt gratitude to everyone who supported us throughout this **Object-Oriented Programming (CE-225L)** project on **Loan Management System**.

First and foremost, we extend our deepest thanks to our parents for their unwavering support and encouragement. Their belief in our abilities has been a constant source of motivation and strength.
We are deeply grateful to our teacher, **Engr. Samina Noreen**, for her valuable guidance, mentorship, and constructive feedback. Her insights were instrumental in shaping the direction and successful completion of this project.

We also wish to acknowledge our friends and classmates for their cooperation, encouragement, and assistance during this journey, which kept us motivated and focused.

Shahzeb Shahid
Mosa Raza
M. Shaheer Asif

15. References

1	**GitHub** **https://github.com/punit3299/loan-application-backend**
2	**OpenAI, "ChatGPT," OpenAI Language Model, accessed May 2025. [Online]. Available:** **https://chat.openai.com/**
3	**Data Flow Diagrams** **https://www.mermaidchart.com/app/dashboard**
4	**Format Source Code:** **Format your source code | Online Tool - deBUG.to**